YouCode
POWERED BY SIMPLON

LES PRINCIPES S.O.L.I.D

AAMIR EL AMIRI

YouCode
POWERED BY SIMPLON

SOLID

Nous n’allons pas parler de solidité mais d’un concept : le principe SOLID (S.O.L.I.D). C’est un acronyme composé de 5 lettres, de 5 principes de bases pour la Programmation Orientée Objet (POO).

QUE PENSEZ-VOUS DE CE QUI EST SOLID ?

PLAN DE PRESENTATION

01

Les cinq principes SOLID

S.O.L.I.D

10%

02

Importance de SOLID

Importance de SOLID dans le développement logiciel

20%

03

Exemples

Exemples théoriques et pratiques pour chaque principe

25%

40%

30%

99%

04

Design Patterns

Relation entre les principes SOLID et les Design Patterns

05

Avantages

Avantages et défis d’application

06

Conclusion

Résumé et discussions

S.O.L.I.D

S — SRP

Single Responsibility Principle

O — OCP

Open/Closed Principle

L — LSP

Liskov's Substitution Principle

I — ISP

Interface Segregation Principle

D — DIP

Dependency Inversion Principle

CREATIVE TIMELINE

S

Single Responsibility Principle

Une classe ne doit avoir qu'une seule et unique responsabilité.

01

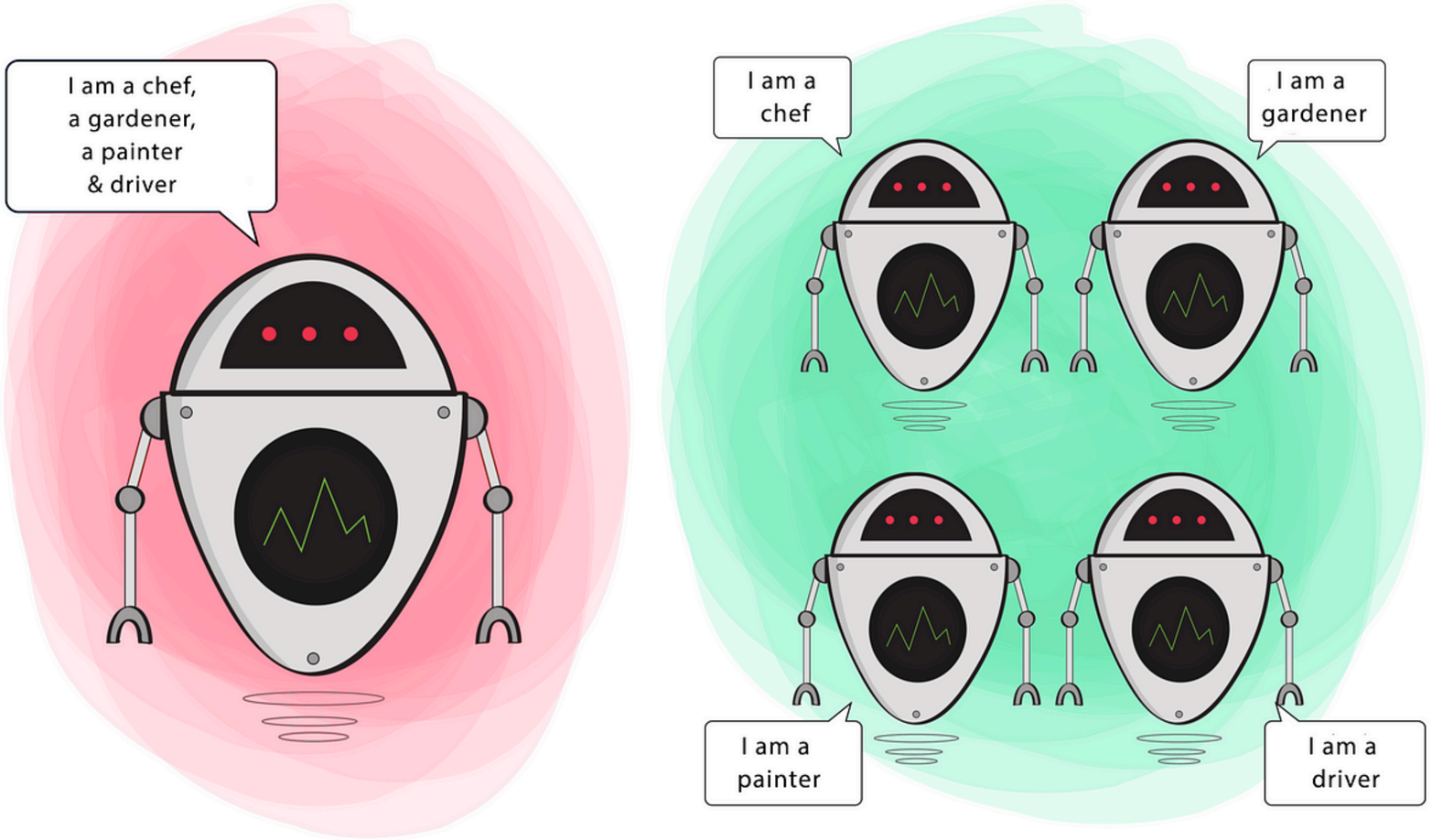
Pourquoi est-ce important ?

Une classe avec plusieurs raisons de changer devient fragile. Séparer les responsabilités rend le code plus stable.

02

Comment l'appliquer ?

Séparez les tâches. Par exemple, créez deux classes : une pour la logique métier et une pour la persistance des données.



Single Responsibility

EXEMPLE PRATIQUE

Mauvaise pratique

```
class User {
    constructor(public nom: string, public age: number) {}

    save() {
        // Code pour sauvegarder l'utilisateur dans une base de données
    }
}
```

Bonne pratique

```
class User {
    constructor(public name: string, public age: number) {}
}

class UserDatabase {
    save(user: User) {
        // Code pour sauvegarder l'utilisateur dans une base de données
    }
}
```

OPEN/CLOSED PRINCIPLE

O

Open/Closed Principle

Les entités doivent être ouvertes à l'extension et fermées à la modification.

01

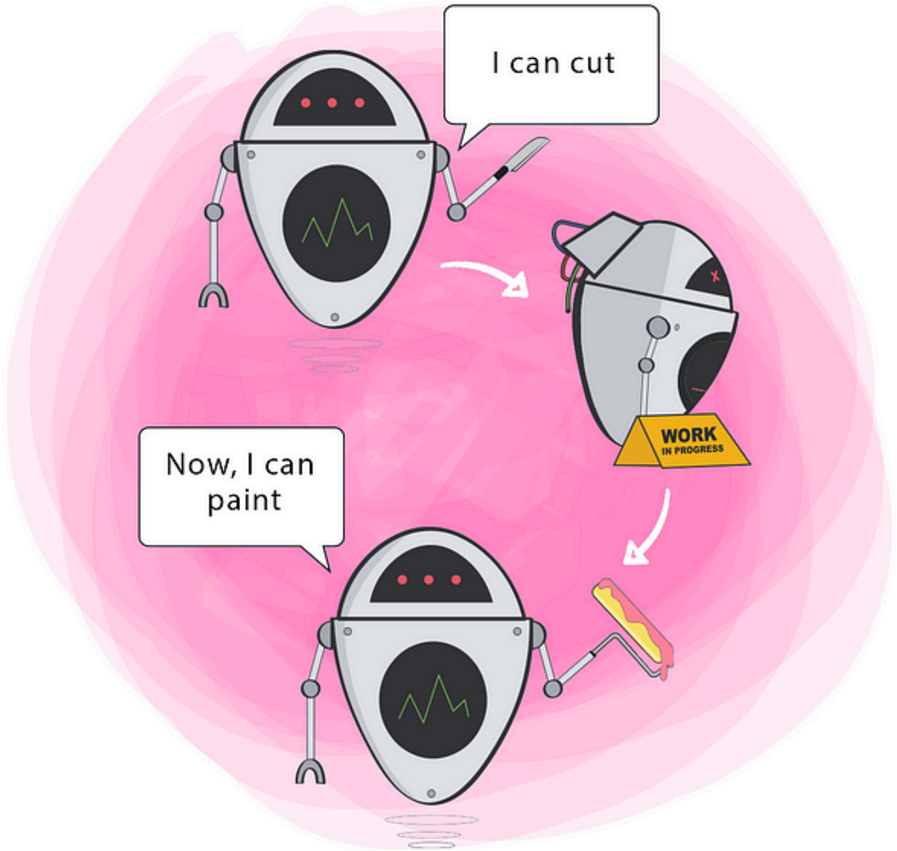
Pourquoi est-ce important ?

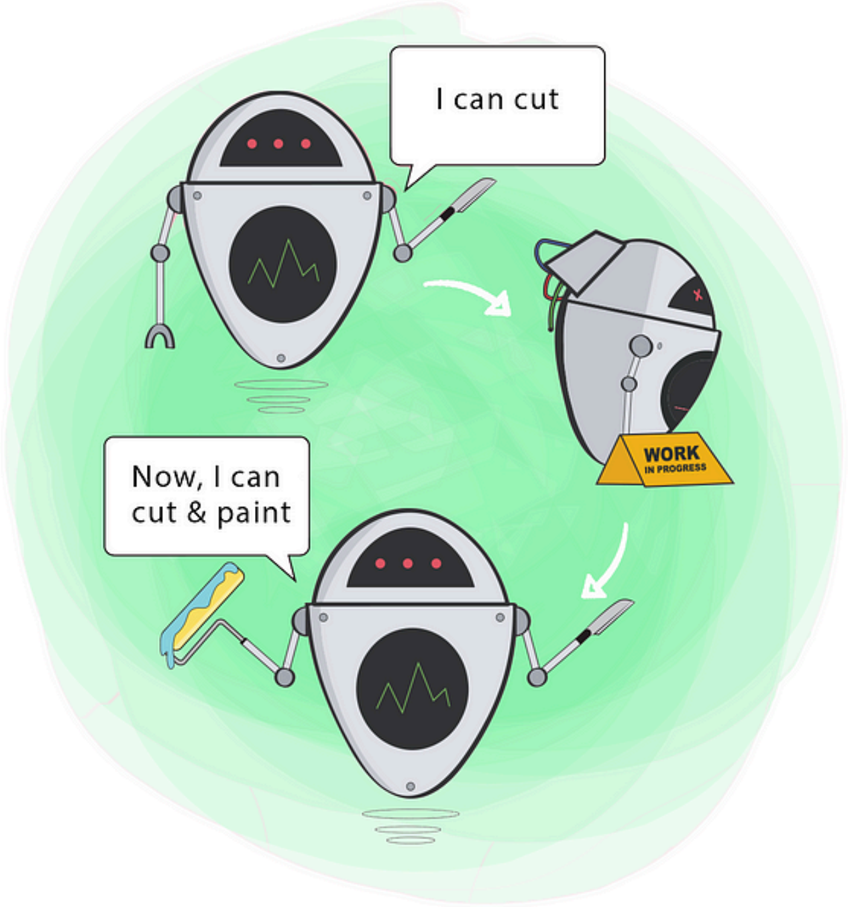
Modifier constamment le code existant augmente le risque de bugs. Ce principe réduit ce risque.

02

Comment l'appliquer ?

Utilisez l'héritage ou les interfaces pour ajouter des fonctionnalités sans toucher au code existant.


I can cut
WORK
IN PROGRESS
Now, I can
paint


I can cut
WORK
IN PROGRESS
Now, I can
cut & paint

Open-Closed

EXEMPLE PRATIQUE

Mauvaise pratique

```
class Rectangle {
    constructor(public width: number, public height: number) {}
}

function area(shapes: any[]): number {
    let total = 0;
    for (const shape of shapes) {
        if (shape instanceof Rectangle) {
            total += shape.width * shape.height;
        }
    }
    return total;
}
```

Bonne pratique

```
interface Shape {
    area(): number;
}

class Rectangle implements Shape {
    constructor(public width: number, public height: number) {}

    area(): number {
        return this.width * this.height;
    }
}

function totalArea(shapes: Shape[]): number {
    return shapes.reduce((acc, shape) => acc + shape.area(), 0);
}
```

DEPENDENCY INVERSION PRINCIPLE

L

Dependency Inversion Principle

01

Pourquoi est-ce important ?

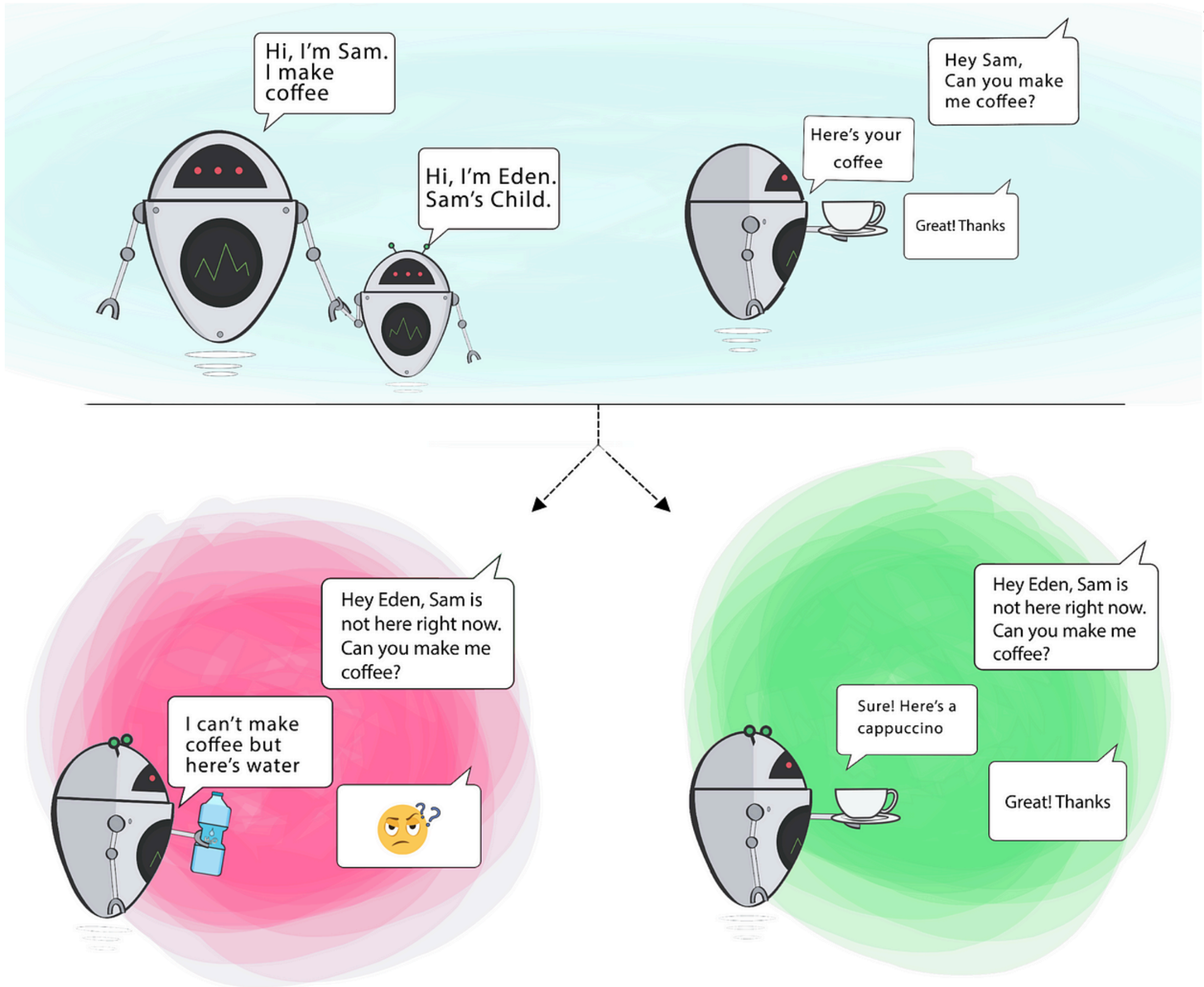
Violer ce principe peut causer des bugs difficiles à détecter dans les classes dérivées.

02

Comment l'appliquer ?

Assurez-vous que les classes dérivées respectent le comportement de la classe de base sans le modifier.

Les objets dans un programme doivent être remplaçables par des instances de leur sous-type sans pour autant altérer le bon fonctionnement du programme.



Liskov Substitution

EXEMPLE PRATIQUE

Mauvaise pratique

```
class Bird {
    fly() {
        // ...
    }
}

class Penguin extends Bird {
    fly() {
        throw new Error("Les pingouins ne peuvent pas voler !");
    }
}
```

Bonne pratique

```
class Bird {
    fly() {
        // ...
    }
}

class FlightlessBird extends Bird {
    fly() {
        console.log("Je ne peux pas voler.");
    }
}

class Penguin extends FlightlessBird {}
```

CREATIVE TIMELINE

I

Interface Segregation Principle

Aucun client ne devrait être forcé d'implémenter des méthodes / fonctions qu'il n'utilise pas.

01

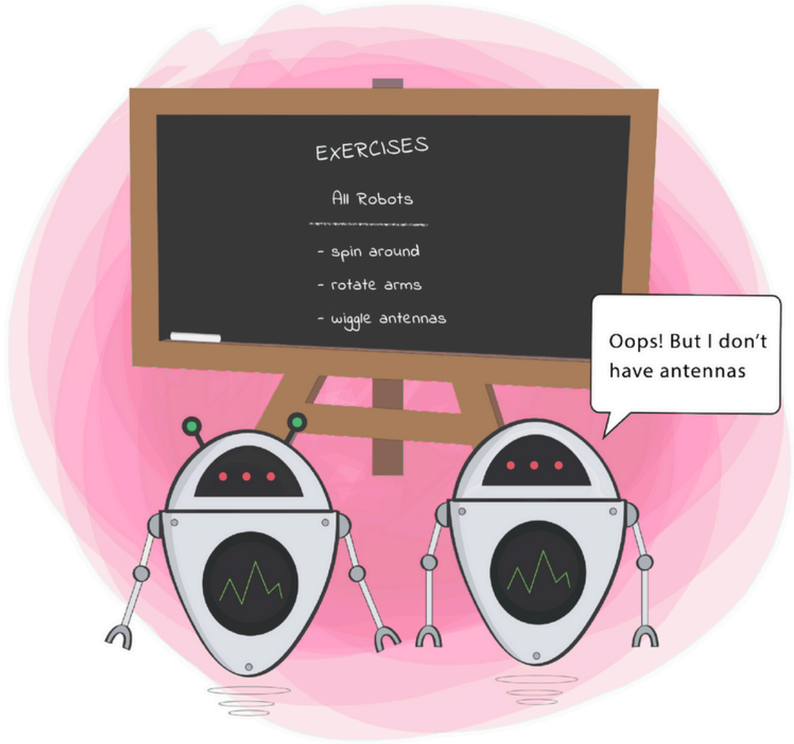
Pourquoi est-ce important ?

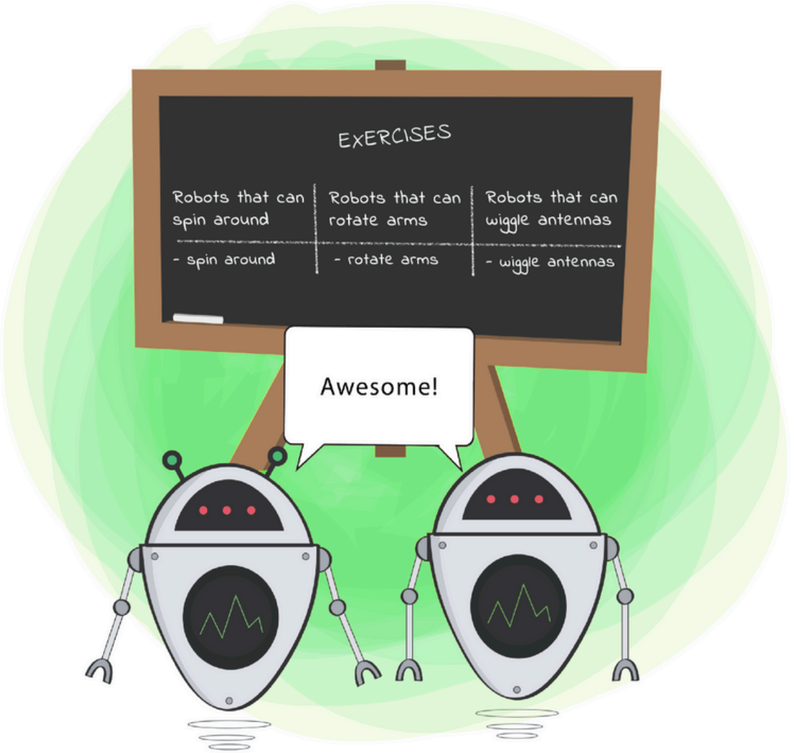
Une interface avec des méthodes inutiles complique l'implémentation et la compréhension du système.

02

Comment l'appliquer ?

Créez des interfaces petites et spécifiques, sans obliger un client à implémenter des méthodes qu'il n'utilise pas.





Interface Segregation

EXEMPLE PRATIQUE

Mauvaise pratique

```
interface Worker {
    work(): void;
    eat(): void;
}
```

Bonne pratique

```
interface Eater {
    eat(): void;
}

interface Worker {
    work(): void;
}
```

CREATIVE TIMELINE

D

Dependency Inversion Principle

Une classe doit dépendre de son abstraction, pas de son implémentation.

01

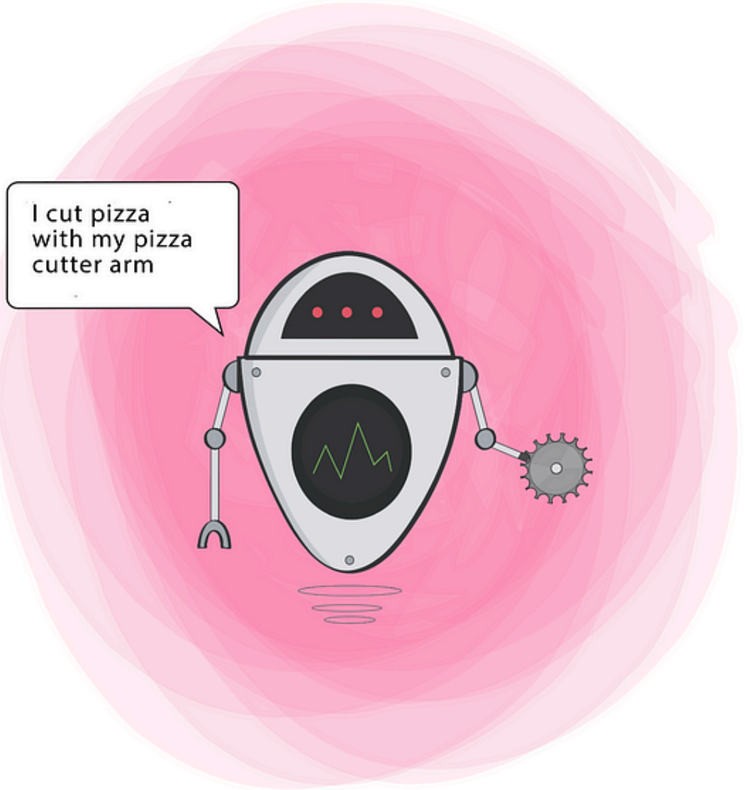
Pourquoi est-ce important ?

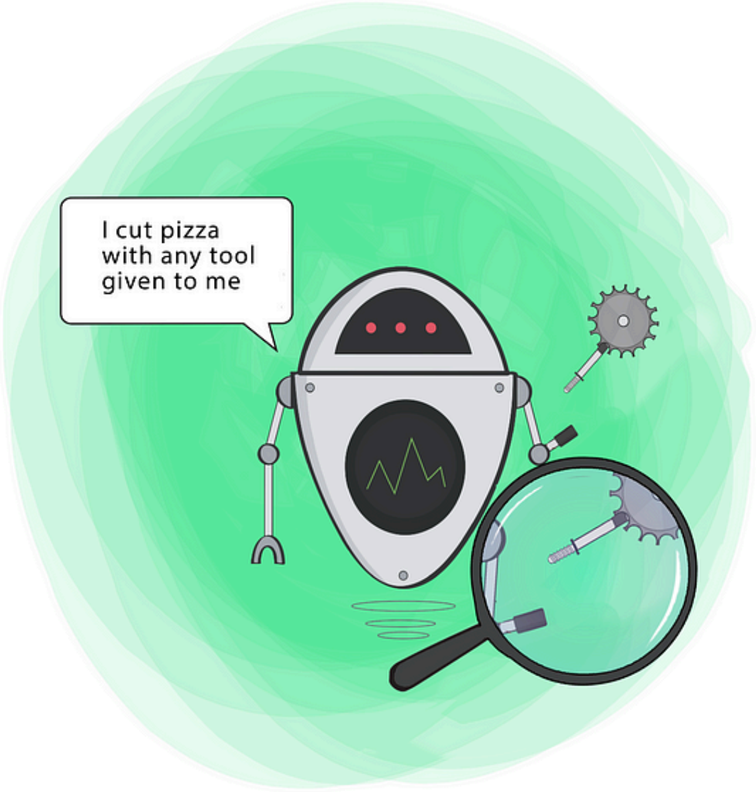
Cela réduit le couplage entre les composants, rendant le système plus modulaire et plus facile à maintenir.

02

Comment l'appliquer ?

Utilisez des abstractions au lieu d'implémentations concrètes et passez-les en paramètre via l'injection de dépendance.





Dependency Inversion

EXEMPLE PRATIQUE

Mauvaise pratique

```
class MySQLDatabase {
    connect() {
        // ...
    }
}

class UserManager {
    db = new MySQLDatabase();
}
```

Bonne pratique

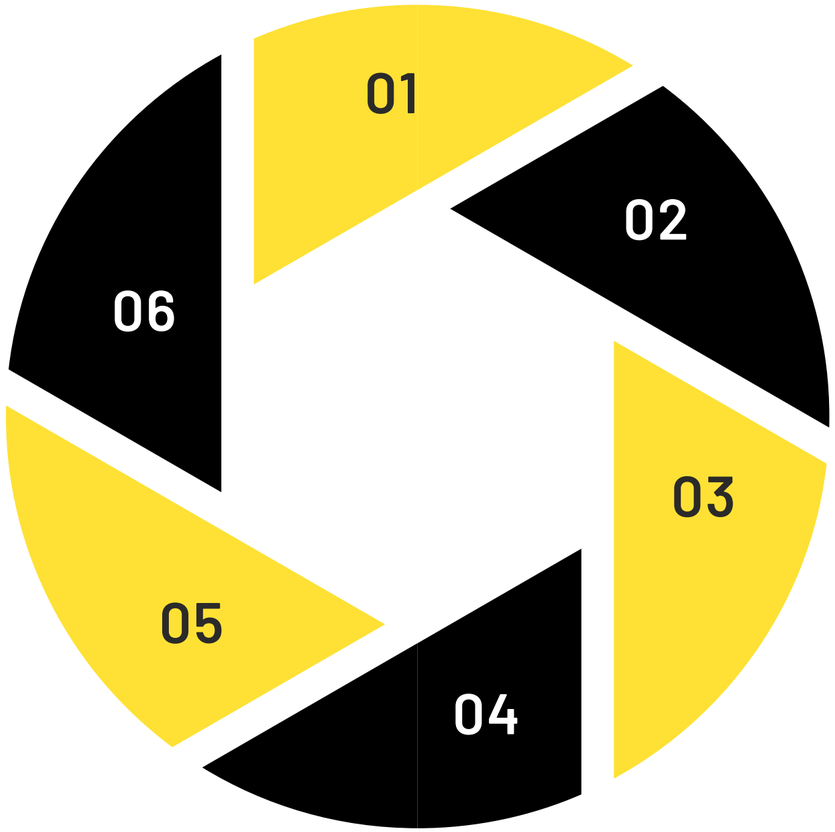
```
interface Database {
    connect(): void;
}

class MySQLDatabase implements Database {
    connect() {
        // ...
    }
}

class UserManager {
    constructor(private database: Database) {}
}
```

IMPORTANCE DE SOLID DANS LE DÉVELOPPEMENT LOGICIEL

SOLID

Les principes **SOLID** sont conçus pour rendre le code plus clair, plus compréhensible et plus facile à modifier. Il ne s'agit pas de règles strictes, mais plutôt de directives pour aider à concevoir un code plus robuste et maintenable. Ils sont particulièrement utiles pour travailler en équipe, où la clarté et la facilité de modification sont cruciales.

AVANTAGES D'APPLICATION



Code testable

Code maintenable

Code plus logique

Code plus lisible

Code moins bogué

Code extensible (tu changes une partie du programme et il continue de fonctionner)

RELATION ENTRE LES PRINCIPES SOLID ET LES DESIGN PATTERNS

Relation entre les principes SOLID et les Design Patterns

Les principes SOLID et les Design Patterns sont liés en ce qu'ils visent à améliorer la structure et la flexibilité du code. SOLID fournit des règles de conception, tandis que les Design Patterns offrent des solutions réutilisables.

Les Design Patterns aident à appliquer les principes SOLID de manière cohérente.

Conclusion

Les principes SOLID offrent un cadre pour développer un code plus propre, plus efficace et plus maintenable. En suivant ces principes, les développeurs peuvent créer des systèmes plus robustes et plus flexibles, tout en probable la probabilité d'introduire des erreurs lors des modifications du code.